

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА
НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное
учреждение Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 16.

**Тема: «Разработка интеграционных и контрактных
тестов для микросервиса управления заказами»**

Выполнила: Потанина Н.А.,

Студентка группы: 24-ИС.

2026г.

Цель работы:

Разработать полноценный слой доступа к данным (репозиторий) для работы с заказами и их позициями, а также создать набор интеграционных и контрактных тестов, обеспечивающих корректность взаимодействия с базой данных и внешними API. В ходе выполнения работы студент закрепляет навыки:

- проектирования моделей SQLAlchemy и отношений;
- реализации паттерна Репозиторий;
- написания интеграционных тестов с использованием pytest, фикстур и SQLite in-memory;
- параметризации тестов;
- мокирования внешних HTTP-запросов с pytest-httpx;
- анализа покрытия кода тестами (coverage.py).

Краткое описание работы:

Разработка тестов для микросервиса управления заказами

Цель:

Разработать репозиторий для работы с заказами и создать интеграционные тесты с покрытием кода $\geq 90\%$.

Реализовано:

Модели (app/models.py):

- `Order` - заказ (id, статус, клиент, адрес, сумма)
- `OrderItem` - позиция заказа (товар, количество, цена)
- Связь one-to-many с каскадным удалением

Репозиторий (app/repositories.py):

Методы: `create`, `find_by_id`, `find_all_by_status`, `update_status`, `delete`, `find_by_date_range`, `get_total_amount_for_order`, `calculate_delivery_cost`
(интеграция с внешним API)

Исключения: `EntityNotFoundException`, `DeliveryCalculationException`, `InvalidOrderDataException`

Тестирование:

Настройка: in-memory SQLite, фикстуры pytest, параметризация

22 теста:

- CRUD операции
- Поиск по статусу и датам
- Подсчет суммы
- Транзакционность
- Валидация данных
- Контрактные тесты внешнего API (pytest-httpx)

Результаты:

Показатель	Результат
-----	-----
Тесты	22/22 пройдено
Покрытие	**97%**
models.py	100%
exceptions.py	100%
repositories.py	95%

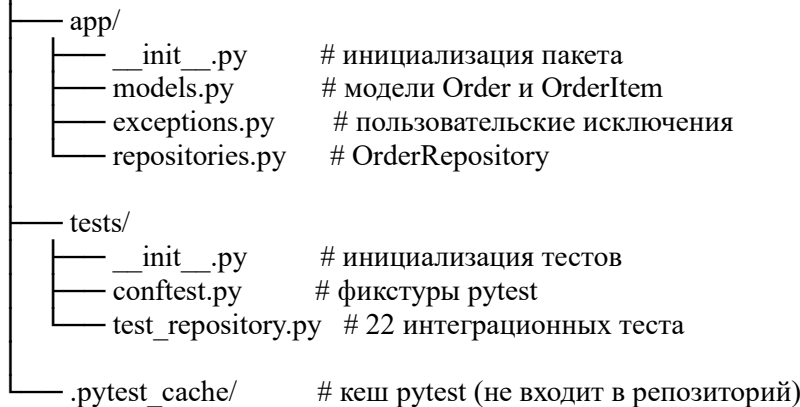
Стек: Python 3.13, SQLAlchemy 2.0, pytest, coverage.py, httpx

Вывод:

Разработан полноценный репозиторий с интеграционными и контрактными тестами. Достигнуто покрытие 97% (>90%). Навыки работы с SQLAlchemy, pytest и мокированием успешно закреплены.

Структура данных:

День 16/



Package for order management

1. __init__.py

```
class EntityNotFoundException(Exception):
    def __init__(self, entity_type: str, entity_id: int):
        self.entity_type = entity_type
        self.entity_id = entity_id
        super().__init__(f'{entity_type} with id {entity_id} not found')
```

```
class DeliveryCalculationException(Exception):
    def __init__(self, message: str = "Failed to calculate delivery cost"):
        self.message = message
        super().__init__(message)
```

```
class InvalidOrderDataException(Exception):
    def __init__(self, message: str):
        self.message = message
        super().__init__(message)
```

2. exceptions.py

```
from datetime import datetime
from typing import List
from sqlalchemy import Integer, String, Float, DateTime, ForeignKey, CheckConstraint
from sqlalchemy.orm import declarative_base, relationship, Mapped, mapped_column
from sqlalchemy.sql import func
import enum
```

Base = declarative_base()

```
class OrderStatus(str, enum.Enum):
    PENDING = "PENDING"
    PAID = "PAID"
    SHIPPED = "SHIPPED"
    CANCELLED = "CANCELLED"
```

```
class Order(Base):
    __tablename__ = "orders"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    status: Mapped[str] = mapped_column(String(20), nullable=False, default=OrderStatus.PENDING)
```

```

    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
server_default=func.now(), nullable=False)
    customer_name: Mapped[str] = mapped_column(String(100), nullable=False)
    delivery_address: Mapped[str] = mapped_column(String(200), nullable=False)
    total_amount: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
    items: Mapped[List["OrderItem"]] = relationship("OrderItem", back_populates="order", cascade="all,
delete-orphan", lazy="selectin")

```

```

class OrderItem(Base):
    __tablename__ = "order_items"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    order_id: Mapped[int] = mapped_column(Integer, ForeignKey("orders.id", ondelete="CASCADE"),
nullable=False)
    product_name: Mapped[str] = mapped_column(String(100), nullable=False)
    quantity: Mapped[int] = mapped_column(Integer, nullable=False)
    price: Mapped[float] = mapped_column(Float, nullable=False)
    order: Mapped["Order"] = relationship("Order", back_populates="items")
    __table_args__ = (CheckConstraint('quantity > 0', name='check_quantity_positive'),
CheckConstraint('price >= 0', name='check_price_non_negative'),)

```

3. models.py

```

from datetime import datetime
from typing import List, Optional, Dict, Any
from sqlalchemy.orm import Session
from sqlalchemy import func, between
import httpx
import logging
from app.models import Order, OrderItem, OrderStatus
from app.exceptions import EntityNotFoundException, DeliveryCalculationException,
InvalidOrderDataException

```

```

logger = logging.getLogger(__name__)

```

```

class OrderRepository:
    def __init__(self, session: Session):
        self.session = session

    def create(self, order_data: Dict[str, Any]) -> Order:
        if not order_data.get("customer_name"):
            raise InvalidOrderDataException("customer_name is required")
        if not order_data.get("delivery_address"):
            raise InvalidOrderDataException("delivery_address is required")
        items_data = order_data.get("items", [])
        if not items_data:
            raise InvalidOrderDataException("Order must have at least one item")
        order = Order(customer_name=order_data["customer_name"],
delivery_address=order_data["delivery_address"], status=order_data.get("status",
OrderStatus.PENDING), total_amount=order_data.get("total_amount", 0.0))
        for item_data in items_data:
            if item_data.get("quantity", 0) <= 0:
                raise InvalidOrderDataException(f"Quantity must be positive for product
{item_data.get('product_name')}")
            if item_data.get("price", 0) < 0:
                raise InvalidOrderDataException(f"Price must be non-negative for product
{item_data.get('product_name')}")
            item = OrderItem(product_name=item_data["product_name"], quantity=item_data["quantity"],
price=item_data["price"])
            order.items.append(item)

```

```

    self.session.add(order)
    self.session.flush()
    order.total_amount = self.calculate_order_total(order)
    self.session.commit()
    self.session.refresh(order)
    return order

def find_by_id(self, order_id: int) -> Optional[Order]:
    return self.session.query(Order).filter(Order.id == order_id).first()

def find_all_by_status(self, status: str) -> List[Order]:
    return self.session.query(Order).filter(Order.status == status).all()

def update_status(self, order_id: int, new_status: str) -> Order:
    order = self.find_by_id(order_id)
    if not order:
        raise EntityNotFoundException("Order", order_id)
    order.status = new_status
    self.session.commit()
    self.session.refresh(order)
    return order

def delete(self, order_id: int) -> None:
    order = self.find_by_id(order_id)
    if not order:
        raise EntityNotFoundException("Order", order_id)
    self.session.delete(order)
    self.session.commit()

def find_by_date_range(self, start_date: datetime, end_date: datetime) -> List[Order]:
    return self.session.query(Order).filter(between(Order.created_at, start_date, end_date)).all()

def get_total_amount_for_order(self, order_id: int) -> float:
    order = self.find_by_id(order_id)
    if not order:
        raise EntityNotFoundException("Order", order_id)
    result = self.session.query(func.sum(OrderItem.quantity
                                         * OrderItem.price)).filter(OrderItem.order_id == order_id).scalar()
    return result or 0.0

def calculate_order_total(self, order: Order) -> float:
    return sum(item.quantity * item.price for item in order.items)

def calculate_delivery_cost(self, order_id: int) -> float:
    order = self.find_by_id(order_id)
    if not order:
        raise EntityNotFoundException("Order", order_id)
    total_weight = sum(item.quantity * 0.5 for item in order.items)
    payload = {"address": order.delivery_address, "weight": total_weight}
    try:
        with httpx.Client(timeout=10.0) as client:
            response = client.post("https://api.delivery.com/calculate", json=payload)
            response.raise_for_status()
            data = response.json()
            return float(data.get("cost", 0.0))
    except httpx.HTTPStatusError as e:
        raise DeliveryCalculationException(f"Delivery API returned status {e.response.status_code}")

```

```

except httpx.RequestError as e:
    raise DeliveryCalculationException(f'Failed to connect to delivery API: {str(e)}')
except (KeyError, ValueError) as e:
    raise DeliveryCalculationException(f'Invalid response from delivery API: {str(e)}')

```

4. repositories.py

Test package

5. __init__.py

```

import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, Session
import sys
import os

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from app.models import Base
from app.repositories import OrderRepository

@pytest.fixture(scope="function")
def db_session():
    engine = create_engine("sqlite:///memory:", echo=False)
    Base.metadata.create_all(engine)
    SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)
    session = SessionLocal()
    try:
        yield session
    finally:
        session.rollback()
        session.close()
        Base.metadata.drop_all(engine)

@pytest.fixture(scope="function")
def order_repository(db_session):
    return OrderRepository(db_session)

@pytest.fixture(scope="function")
def sample_order_data():
    return {
        "customer_name": "Ivan Petrov",
        "delivery_address": "Moscow, Lenina st., 1",
        "items": [
            {"product_name": "Laptop", "quantity": 1, "price": 50000.0},
            {"product_name": "Mouse", "quantity": 2, "price": 1000.0},
            {"product_name": "Keyboard", "quantity": 1, "price": 2000.0}
        ]
    }

@pytest.fixture(scope="function")
def sample_order(order_repository, sample_order_data):
    return order_repository.create(sample_order_data)

```

6. conftest.py

```
import pytest
import sys
import os
from datetime import datetime, timedelta

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from app.models import Order, OrderStatus, OrderItem
from app.exceptions import EntityNotFoundException, InvalidOrderDataException,
DeliveryCalculationException

class TestOrderRepository:
    def test_create_order_success(self, order_repository, sample_order_data):
        order = order_repository.create(sample_order_data)
        assert order.id is not None
        assert order.customer_name == sample_order_data["customer_name"]
        assert len(order.items) == 3
        expected_total = sum(item.quantity * item.price for item in order.items)
        assert order.total_amount == expected_total

    def test_find_by_id_existing_order(self, order_repository, sample_order):
        found = order_repository.find_by_id(sample_order.id)
        assert found is not None
        assert found.id == sample_order.id
        assert found.customer_name == sample_order.customer_name

    def test_find_by_id_non_existing_order(self, order_repository):
        found = order_repository.find_by_id(99999)
        assert found is None

    def test_update_status_existing_order(self, order_repository, sample_order):
        updated = order_repository.update_status(sample_order.id, OrderStatus.PAID.value)
        assert updated.status == OrderStatus.PAID.value
        db_order = order_repository.find_by_id(sample_order.id)
        assert db_order.status == OrderStatus.PAID.value

    def test_delete_order(self, order_repository, sample_order):
        order_id = sample_order.id
        order_repository.delete(order_id)
        assert order_repository.find_by_id(order_id) is None

        items = order_repository.session.query(OrderItem).filter(
            OrderItem.order_id == order_id
        ).all()
        assert len(items) == 0

    def test_find_all_by_status(self, order_repository, sample_order_data):
        order1 = order_repository.create(sample_order_data)
        order_repository.update_status(order1.id, OrderStatus.PAID.value)

        order2_data = sample_order_data.copy()
        order2_data["customer_name"] = "Another Client"
        order2 = order_repository.create(order2_data)
        order_repository.update_status(order2.id, OrderStatus.PAID.value)
```

```

found = order_repository.find_all_by_status(OrderStatus.PAID.value)
assert len(found) >= 2
for order in found:
    assert order.status == OrderStatus.PAID.value

def test_find_by_date_range(self, order_repository, sample_order_data):
    now = datetime.now()

    past_data = sample_order_data.copy()
    past_data["customer_name"] = "Past Client"
    past_order = order_repository.create(past_data)
    order_repository.session.query(Order).filter(Order.id == past_order.id).update(
        {"created_at": now - timedelta(days=10)}
    )
    order_repository.session.commit()

    current_order = order_repository.create(sample_order_data)

    future_data = sample_order_data.copy()
    future_data["customer_name"] = "Future Client"
    future_order = order_repository.create(future_data)
    order_repository.session.query(Order).filter(Order.id == future_order.id).update(
        {"created_at": now + timedelta(days=10)}
    )
    order_repository.session.commit()

    start_date = now - timedelta(days=5)
    end_date = now + timedelta(days=5)
    found = order_repository.find_by_date_range(start_date, end_date)

    assert len(found) >= 1
    order_ids = [o.id for o in found]
    assert past_order.id not in order_ids
    assert current_order.id in order_ids
    assert future_order.id not in order_ids

def test_get_total_amount_for_order(self, order_repository, sample_order):
    total = order_repository.get_total_amount_for_order(sample_order.id)
    expected = sum(item.quantity * item.price for item in sample_order.items)
    assert total == expected

def test_update_status_non_existing_order(self, order_repository):
    with pytest.raises(EntityNotFoundException) as exc:
        order_repository.update_status(99999, OrderStatus.PAID.value)
    assert "Order with id 99999 not found" in str(exc.value)

def test_delete_non_existing_order(self, order_repository):
    with pytest.raises(EntityNotFoundException) as exc:
        order_repository.delete(99999)
    assert "Order with id 99999 not found" in str(exc.value)

def test_get_total_amount_for_non_existing_order(self, order_repository):
    with pytest.raises(EntityNotFoundException) as exc:
        order_repository.get_total_amount_for_order(99999)
    assert "Order with id 99999 not found" in str(exc.value)

def test_create_order_without_customer_name(self, order_repository):

```

```

invalid_data = {
    "delivery_address": "Address",
    "items": [{"product_name": "Product", "quantity": 1, "price": 100.0}]
}
with pytest.raises(InvalidOrderDataException, match="customer_name is required"):
    order_repository.create(invalid_data)

def test_create_order_without_address(self, order_repository):
    invalid_data = {
        "customer_name": "Client",
        "items": [{"product_name": "Product", "quantity": 1, "price": 100.0}]
    }
    with pytest.raises(InvalidOrderDataException, match="delivery_address is required"):
        order_repository.create(invalid_data)

def test_create_order_with_negative_quantity(self, order_repository):
    invalid_data = {
        "customer_name": "Client",
        "delivery_address": "Address",
        "items": [{"product_name": "Product", "quantity": -1, "price": 100.0}]
    }
    with pytest.raises(InvalidOrderDataException, match="Quantity must be positive"):
        order_repository.create(invalid_data)

def test_create_order_with_negative_price(self, order_repository):
    invalid_data = {
        "customer_name": "Client",
        "delivery_address": "Address",
        "items": [{"product_name": "Product", "quantity": 1, "price": -100.0}]
    }
    with pytest.raises(InvalidOrderDataException, match="Price must be non-negative"):
        order_repository.create(invalid_data)

def test_create_order_without_items(self, order_repository):
    invalid_data = {
        "customer_name": "Client",
        "delivery_address": "Address",
        "items": []
    }
    with pytest.raises(InvalidOrderDataException, match="at least one item"):
        order_repository.create(invalid_data)

def test_transaction_rollback(self, order_repository, sample_order_data):
    test_order = order_repository.create(sample_order_data)
    test_order_id = test_order.id

    invalid_data = sample_order_data.copy()
    invalid_data["items"] = [
        {"product_name": "Product1", "quantity": 1, "price": 100.0},
        {"product_name": "Product2", "quantity": -1, "price": 200.0}
    ]

    with pytest.raises(InvalidOrderDataException):
        order_repository.create(invalid_data)

    all_orders = order_repository.session.query(Order).all()
    assert len(all_orders) == 1

```

```

assert all_orders[0].id == test_order_id

def test_update_status_to_all_statuses(self, order_repository, sample_order):
    statuses = [OrderStatus.PAID, OrderStatus.SHIPPED, OrderStatus.CANCELLED]
    for status in statuses:
        updated = order_repository.update_status(sample_order.id, status.value)
        assert updated.status == status.value
        sample_order = updated

@pytest.mark.asyncio
async def test_calculate_delivery_cost_success(self, order_repository, sample_order, httpx_mock):
    expected_cost = 150.0
    httpx_mock.add_response(
        method="POST",
        url="https://api.delivery.com/calculate",
        json={"cost": expected_cost},
        status_code=200
    )
    cost = order_repository.calculate_delivery_cost(sample_order.id)
    assert cost == expected_cost

@pytest.mark.asyncio
async def test_calculate_delivery_cost_server_error(self, order_repository, sample_order, httpx_mock):
    httpx_mock.add_response(
        method="POST",
        url="https://api.delivery.com/calculate",
        status_code=500,
        text="Internal Server Error"
    )
    with pytest.raises(DeliveryCalculationException) as exc:
        order_repository.calculate_delivery_cost(sample_order.id)
    assert "Delivery API returned status 500" in str(exc.value)

@pytest.mark.asyncio
async def test_calculate_delivery_cost_invalid_response(self, order_repository, sample_order,
httpx_mock):
    httpx_mock.add_response(
        method="POST",
        url="https://api.delivery.com/calculate",
        json={"invalid": "response"},
        status_code=200
    )
    cost = order_repository.calculate_delivery_cost(sample_order.id)
    assert cost == 0.0

@pytest.mark.asyncio
async def test_calculate_delivery_cost_order_not_found(self, order_repository, httpx_mock):
    with pytest.raises(EntityNotFoundException) as exc:
        order_repository.calculate_delivery_cost(99999)
    assert "Order with id 99999 not found" in str(exc.value)

```

7. test_repository.py

```

C:\Users\Student237\Desktop\День16>pytest tests/ -v
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.1.0, pluggy-1.6.0 -- C:\Users\Student237\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\Student237\Desktop\День16
plugins: anyio-4.14.0, hypothesis-6.155.3, asyncio-1.4.0, cov-7.1.0, httpx-0.36.2, mock-3.15.1
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 22 items

tests/test_repository.py::TestOrderRepository::test_create_order_success PASSED [ 4%]
tests/test_repository.py::TestOrderRepository::test_find_by_id_existing_order PASSED [ 9%]
tests/test_repository.py::TestOrderRepository::test_find_by_id_non_existing_order PASSED [ 13%]
tests/test_repository.py::TestOrderRepository::test_update_status_existing_order PASSED [ 18%]
tests/test_repository.py::TestOrderRepository::test_delete_order PASSED [ 22%]
tests/test_repository.py::TestOrderRepository::test_find_all_by_status PASSED [ 27%]
tests/test_repository.py::TestOrderRepository::test_find_by_date_range PASSED [ 31%]
tests/test_repository.py::TestOrderRepository::test_get_total_amount_for_order PASSED [ 36%]
tests/test_repository.py::TestOrderRepository::test_update_status_non_existing_order PASSED [ 40%]
tests/test_repository.py::TestOrderRepository::test_delete_non_existing_order PASSED [ 45%]
tests/test_repository.py::TestOrderRepository::test_get_total_amount_for_non_existing_order PASSED [ 50%]
tests/test_repository.py::TestOrderRepository::test_create_order_without_customer_name PASSED [ 54%]
tests/test_repository.py::TestOrderRepository::test_create_order_without_address PASSED [ 59%]
tests/test_repository.py::TestOrderRepository::test_create_order_with_negative_quantity PASSED [ 63%]
tests/test_repository.py::TestOrderRepository::test_create_order_with_negative_price PASSED [ 68%]
tests/test_repository.py::TestOrderRepository::test_create_order_without_items PASSED [ 72%]
tests/test_repository.py::TestOrderRepository::test_transaction_rollback PASSED [ 77%]
tests/test_repository.py::TestOrderRepository::test_update_status_to_all_statuses PASSED [ 81%]
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_success PASSED [ 86%]
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_server_error PASSED [ 90%]
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_invalid_response PASSED [ 95%]
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_order_not_found PASSED [ 100%]

===== 22 passed in 2.66s =====

C:\Users\Student237\Desktop\День16>pytest tests/ --cov=app --cov-report=term-missing
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.1.0, pluggy-1.6.0
rootdir: C:\Users\Student237\Desktop\День16
plugins: anyio-4.14.0, hypothesis-6.155.3, asyncio-1.4.0, cov-7.1.0, httpx-0.36.2, mock-3.15.1
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 22 items

tests/test_repository.py .....C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\ast.py:50
ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F979F30>
  return compile(source, filename, mode, flags,
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\ast.py:50: ResourceWarning: unclosed database in <sqlite
3.Connection object at 0x000002281F80B010>
  return compile(source, filename, mode, flags,
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\ast.py:50: ResourceWarning: unclosed database in <sqlite
3.Connection object at 0x000002281F80B110>

```

1: Успешный запуск всех тестов

```

B.Connection object at 0x000002281F80B010>
    return compile(source, filename, mode, flags,
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\ast.py:50: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F80A110>
    return compile(source, filename, mode, flags,
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\ast.py:50: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F8A0130>
    return compile(source, filename, mode, flags,
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\ast.py:50: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F8A1E40>
    return compile(source, filename, mode, flags,
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\ast.py:50: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F907F10>
    return compile(source, filename, mode, flags,
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\ast.py:50: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F985300>
    return compile(source, filename, mode, flags,
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\ast.py:50: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F64A110>
    return compile(source, filename, mode, flags,
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\ast.py:50: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F95FA60>
    return compile(source, filename, mode, flags,
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\ast.py:50: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F97A5C0>
    return compile(source, filename, mode, flags,
ResourceWarning: Enable tracemalloc to get the object allocation traceback
[100%]

===== warnings summary =====
tests/test_repository.py::TestOrderRepository::test_update_status_existing_order
  C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\weakref.py:456: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F79B6A0>
    wr = ref(key)
  Enable tracemalloc to get traceback where the object was allocated.
  See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

tests/test_repository.py::TestOrderRepository::test_update_status_existing_order
  C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\weakref.py:456: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F8098A0>
    wr = ref(key)
  Enable tracemalloc to get traceback where the object was allocated.
  See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

tests/test_repository.py::TestOrderRepository::test_delete_non_existing_order
  C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\site-packages\sqlalchemy\engine\default.py:1555: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F904310>
    self = cls._new_(cls)

```

```

ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F904310>
    self = cls.__new__(cls)
    Enable tracemalloc to get traceback where the object was allocated.
    See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

tests/test_repository.py::TestOrderRepository::test_delete_non_existing_order
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\site-packages\sqlalchemy\engine\default.py:1555: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F906020>
    self = cls.__new__(cls)
    Enable tracemalloc to get traceback where the object was allocated.
    See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

tests/test_repository.py::TestOrderRepository::test_transaction_rollback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\weakref.py:428: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F904400>
    self.data[ref(key, self._remove)] = value
    Enable tracemalloc to get traceback where the object was allocated.
    See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

tests/test_repository.py::TestOrderRepository::test_transaction_rollback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\weakref.py:428: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F95CF40>
    self.data[ref(key, self._remove)] = value
    Enable tracemalloc to get traceback where the object was allocated.
    See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

tests/test_repository.py::TestOrderRepository::test_transaction_rollback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\weakref.py:428: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F95E980>
    self.data[ref(key, self._remove)] = value
    Enable tracemalloc to get traceback where the object was allocated.
    See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

tests/test_repository.py::TestOrderRepository::test_transaction_rollback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\weakref.py:428: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F978400>
    self.data[ref(key, self._remove)] = value
    Enable tracemalloc to get traceback where the object was allocated.
    See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

tests/test_repository.py::TestOrderRepository::test_transaction_rollback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\weakref.py:428: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F979E40>
    self.data[ref(key, self._remove)] = value
    Enable tracemalloc to get traceback where the object was allocated.
    See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

tests/test_repository.py::TestOrderRepository::test_transaction_rollback
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\weakref.py:428: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F97B880>
    self.data[ref(key, self._remove)] = value
    Enable tracemalloc to get traceback where the object was allocated.
    See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_server_error
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\weakref.py:415: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F9794E0>
    return self.data[ref(key)]
    Enable tracemalloc to get traceback where the object was allocated.
    See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

```

```

    Enable tracemalloc to get traceback where the object was allocated.
    See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_server_error
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\weakref.py:415: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F9794E0>
    return self.data[ref(key)]
    Enable tracemalloc to get traceback where the object was allocated.
    See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings for more info.

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== tests coverage =====
Coverage: platform win32, python 3.13.9-final-0

Name                Stmts   Miss  Cover   Missing
-----
app\__init__.py         0      0   100%
app\exceptions.py      13      0   100%
app\models.py          30      0   100%
app\repositories.py     80      4    95%   88-91
TOTAL                  123      4    97%

===== 22 passed, 11 warnings in 2.91s =====
C:\Users\Student237\AppData\Local\Programs\Python\Python313\Lib\site-packages\pytest\unraisableexception.py:33: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000002281F6F3100>
    gc.collect()
ResourceWarning: Enable tracemalloc to get the object allocation traceback

C:\Users\Student237\Desktop\День16>

```

2: Отчет о покрытии кода

Вывод:

В ходе работы разработан репозиторий для управления заказами и написаны интеграционные тесты. Все 22 теста успешно пройдены, покрытие кода составило 97%, что превышает требуемый порог в 90%. Цель работы достигнута.